Python Engineering - Class 05 Theory Guide

Lists, Tuples and Trajectories

This guide explains the programming, mathematical and/or engineering theory behind the exercise. The goal is to understand the model before memorizing syntax.

1. Why collections are needed

A simulation produces not one state but a sequence of states over time.

Relationship / equation

```
trajectory = [state_0, state_1, ..., state_N]
```

Connection to code

Lists provide dynamic growth while the number of steps may be unknown before the simulation ends.

2. State snapshots

Each tuple stores the values at one instant. The tuple represents a snapshot, not the entire evolving simulation.

Relationship / equation

```
state_k=(h_k,v_k,t_k)
```

Connection to code

Appending a fresh tuple each iteration prevents later updates from overwriting historical states.

3. Mutability and references

Python variables refer to objects. Mutating an object through one reference is visible through every name pointing to that same object.

Relationship / equation

```
a=b -> same object
```

Connection to code

This is why a tuple containing a list does not magically make the inner list immutable.

4. Data extraction

Once trajectory is a collection, engineering quantities can be derived from it instead of being tracked independently.

Relationship / equation

```
max height=max(state[0]); final velocity=trajectory[-1][1]
```

Connection to code

This prepares the transition to NumPy arrays and vectorized data processing.

Manual validation method

For any exercise in this class, use the following protocol before trusting the output:

- 1. Write input values with units or types.
- 2. Predict sign, scale, type and shape of the result.
- 3. Manually calculate a small case when possible.
- 4. Run the code and compare against the prediction.
- 5. Change one input in a controlled way and explain the effect.

Connection to the rest of the course

Trajectory collections become NumPy matrices, files, plots and data pipelines.

Final summary

```
MODEL / CONCEPT
      |
      v
ALGORITHM
      |
      v
PYTHON IMPLEMENTATION
      |
      v
REFERENCE CHECK
      |
      v
ENGINEERING INTERPRETATION
```

> Syntax is the tool. The transferable skill is being able to model, structure, validate and interpret.